

# LD Triggerz

## How to Guide

v1.0.3 - Foundry VTT v13 / v14 - System Agnostic

This guide walks you through installing the module, opening the GM Hub, building your first condition, setting up a trigger, and linking the two together so your table runs without you having to stop the game to manage status effects.

### 1. Installation

LD Triggerz is a release from Lisa's Dungeon. Download the zip from the release post and install it through Foundry's module manager.

Steps:

Log into Foundry and open the Add-on Modules tab.

Click Install Module.

At the bottom of the dialog, paste the manifest URL or click the upload icon to install from the zip directly.

Once installed, enable LD Triggerz in your world's Module Management settings.

Reload the page when prompted.

The module works on any game system. There are no dependencies.

### 2. Opening the GM Hub

Everything in LD Triggerz runs through the GM Hub. You have two ways to open it.

**Scene Control Button:** Look at the left-side scene controls toolbar in Foundry. There is a bolt icon labeled Open LD Triggerz. Click it to open the hub. If you don't see it, check that the scene control is enabled in Configure Settings under the LD Triggerz section.

**Configure Settings:** Open Configure Settings, find the LD Triggerz section, and click Open GM Hub. This works even if you have the scene control button turned off.

The hub shows a live count of your triggers, conditions, and currently selected tokens at the top.

### 3. Understanding the Layout

The GM Hub has four main areas:

**Selected Tokens** - shows tokens you have selected on the canvas, their assigned conditions, and buttons to apply, remove, toggle, assign, or unassign conditions on them directly.

Condition Builder - a form on the left side for creating and saving custom conditions.

Trigger Builder - a form on the right side for creating rules that fire when actor or token data changes.

Configured Conditions and Configured Triggers - the lists at the bottom showing everything you have saved, with delete buttons.

Advanced Import / Export is a collapsible section at the very bottom for backing up and sharing your setup as JSON.

#### 4. Building a Condition

A condition in LD Triggerz is the thing that gets applied to an actor. It can be a native Foundry status effect, a Custom System Builder status, or a fully custom homebrew condition with its own `ActiveEffect` changes.

Using a Foundry or CSB Status: If the condition you want already exists as a Foundry status effect or CSB status, just pick it from the Foundry/CSB Status dropdown. The ID and name fill in automatically. Hit Save Condition and you're done.

Creating a Custom Condition:

Leave the status dropdown on Custom status.

Type a Condition ID - keep it lowercase with hyphens, for example `bloodied` or `near-death`. This is the internal key.

Type a Condition Name - this is what shows in the UI, for example `Bloodied`.

Set an Icon Path if you want a specific icon. The default is `icons/svg/aura.svg`.

Add a Description if it helps you remember what the condition does.

Click Save Condition.

Advanced Effect Changes: Expand the Advanced Effect Changes section if you want the condition to modify actor data through Foundry's `ActiveEffect` system. Each row needs:

Key - the actor data path to modify, for example `system.attributes.hp.value`

Mode - Add, Multiply, Override, Downgrade, Upgrade, or Custom

Value - the number or path to use. For CSB paths like `system.props.HP`, the math is handled automatically.

Priority - controls the order when multiple effects stack. Default is 20.

CSB users: entering `system.props.HP` as the key and a number as the value with Add mode will automatically produce the correct CSB formula. You do not need to write the formula yourself.

#### 5. Building a Trigger

A trigger is a rule that watches for a specific change in actor or token data. When the condition is met, it fires and runs whatever action you assigned.

The Fields:

Trigger Name - a label for your own reference, for example HP Zero Check.

Trigger ID - auto-generated from the name and path if you leave it blank. You can set your own.

Actor Path - the data path to watch. Pick a common one from the dropdown or type a custom path.

Custom Path - if you type here it overrides the dropdown. Use dot notation, for example system.hp.value.

Operator - how to compare the value. Options are Equals, Does not equal, Less than, Less than or equal, Greater than, Greater than or equal.

Value - what to compare against. Can be a number, a percentage like 50%, or another actor data path.

Compare Path - used with percentage values. If your value is 50% and your compare path is system.hp.max, the trigger fires when HP is below half of max.

Scope - All actors, PC actors only, or NPC actors only.

Ignore Zero - tick this to stop the trigger from firing when the value is exactly 0. Useful for preventing loops on death.